

Atina JD Edwards Connector for Mule 4 - User Guide

Table of Contents

Introduction	1
Prerequisites	2
Requirements	3
Preliminary Setup	3
Preparing Maven for Atina JDE Connector	3
Adding Atina JDE license to your Mule runtime	3
AnyPoint Studio Project - Atina JDE Connector	4
Using the Connector	4
Creating a new Mule Project	4
Project Dependencies	5
Repository Configuration	5
Dependency Declaration	5
Other Considerations	6
Troubleshooting	6
Configure the Global Element	7
Set Payload Output	16
Testing the Mule Flow	18
PortX JDE Connector – Demo Applications	19
Example Project	19
Additional Demos	19
Appendix A : Required JARs by Tools Release	19
Tools Release 8.98	19
Tools Releases 9.1 prior Update 4	20
Tools Releases 9.1 Update 4 and later updates	21
Tools Releases 9.2 and later	22
Appendix B : Additional Information	23
Common Call Business Function parameters	23
Handling Errors	24
Defining Data Selection	25

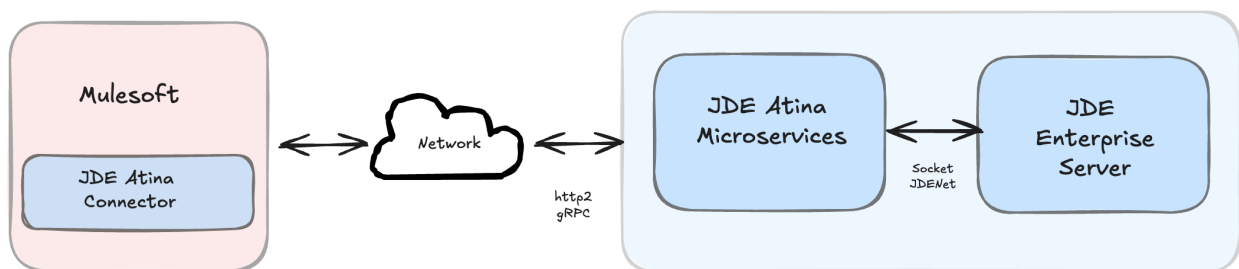
Introduction

This document provides a comprehensive guide to the MuleSoft Connector for Atina JDE

Microservices. It is intended for integration developers and system architects responsible for configuring, deploying, and maintaining integrations between MuleSoft™ and Oracle JD Edwards EnterpriseOne™ using Atina's microservice-based architecture.

Unlike traditional approaches that rely on legacy libraries or complex setups, this connector simplifies the interaction with JD Edwards by leveraging a modern, containerized middleware that exposes a gRPC-based API. The connector encapsulates the complexity of JD Edwards integration and allows Mule applications to seamlessly: After reading this guide, you should be able to accomplish the following scenarios:

1. Log into *JD Edwards EnterpriseOne™*
2. Invoke Web Services (WS) on the server by passing it parameters.
3. Invoke a Business Function (BSFN) on the server by passing it parameters.
4. Invoke a Business Function (BSFN) on the server using Transaction Processing.
5. Invoke a UBE by name.
6. Check a UBE call's status asynchronously.
7. MBF Poll Events.
8. Log out.



At design time, the *JDE Atina Connector* allows you to discover BSFN functions, Web Services by name providing full access to all Business Function and Web Services signatures. Given that discovery is a heavy duty process, the Connector manages a cache repository for all function's metadata used at least once.

The results of a function call will be injected into the flow as a Map containing key-value pairs with the invocation's output values. This approach gives the end user freedom of choice to construct complex flows and manage custom exception strategies.

Prerequisites

This document assumes that the reader has prior knowledge and access to the following technologies and tools:

- Basic understanding of *JD Edwards EnterpriseOne™* concepts (e.g., BSFNs, UBEs, orchestration, data structures).
- Familiarity with MuleSoft development, particularly using [Anypoint Studio](#) for building and deploying integration flows.

Requirements

- **Mule 4.6 Server Runtime** or higher
- **Java JDK 1.8, 11 or 17**
- **AnyPoint Studio™ 7.x or higher**
- **Maven** 3.9.6 or higher (do not use the embedded Maven for AnyPoint Studio)
- **Atina JDE Microservices:** The connector requires access to the Atina JDE Microservices deployed in the target network. Ensure the service is reachable (credentials provided).

Preliminary Setup

Before using the Atina JDE Connector in your MuleSoft projects, some initial configuration is required.

This includes configuring your Maven settings to access the private repository, and installing the license required by the connector.

Preparing Maven for Atina JDE Connector

1. To download the connector and its dependencies, you must add credentials to your Maven settings.xml file (typically located at ~/.m2/settings.xml)
 - a. Locate or create the <servers> section inside the file.
 - b. Add the following entry, replacing youruser and yourpassword with the credentials provided by Atina:

```
<server>
  <id>atina-central</id>
  <username>youruser</username>
  <password>yourpassword</password>
</server>
```

Adding Atina JDE license to your Mule runtime

The *Atina JDE Connector* requires a valid license file. You can provide the license in one of the following ways:

- Place the license file in your project folder under *src/main/resources*.
- Or, copy the license file to the Mule installation directory under *MULE_HOME/conf*.

AnyPoint Studio Project - *Atina JDE Connector*

NOTE

It is recommended that you update AnyPoint Studio before starting with a *Atina JDE Connector* project.

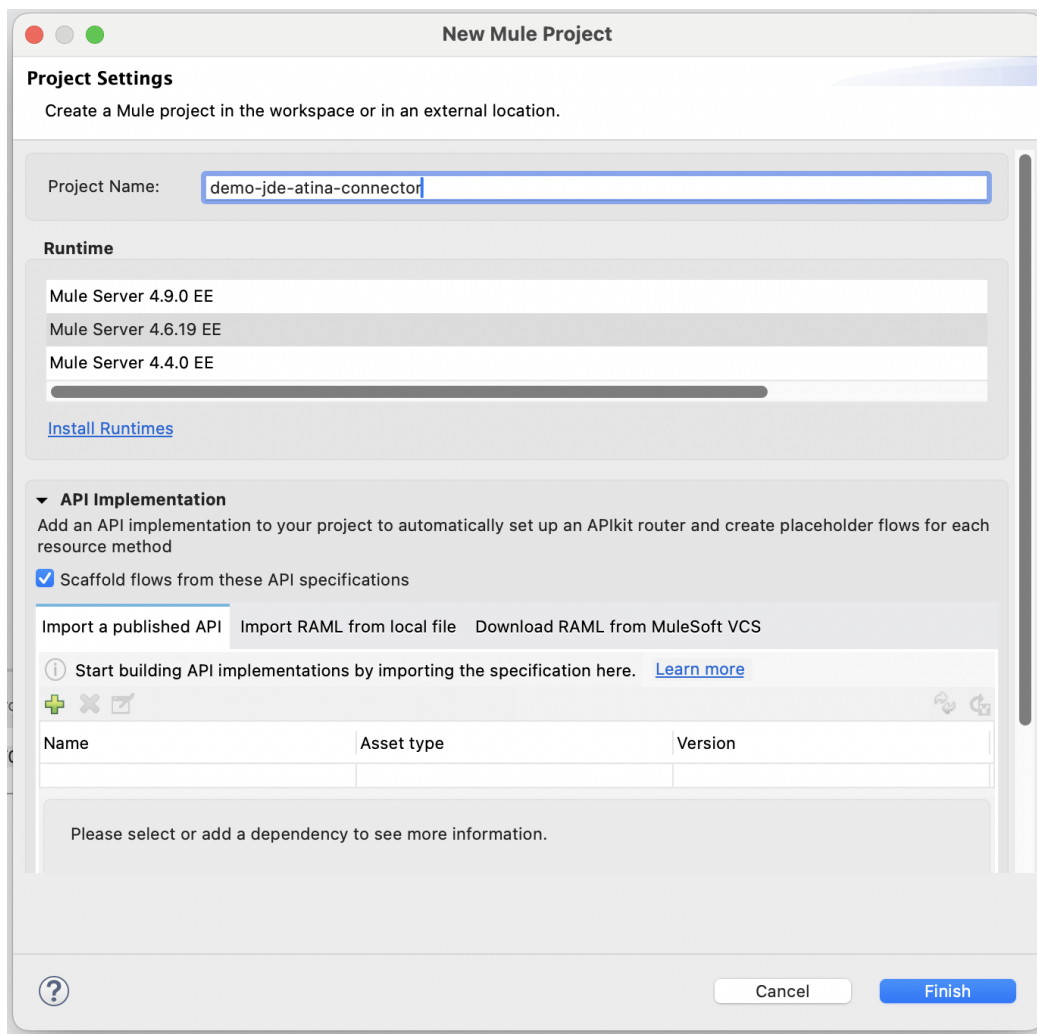
Using the Connector

You can use the connector to

- Log into *JD Edwards EnterpriseOne™*
- Invoke Web Services (WS) on the server by passing it parameters.
- Invoke a Business Function (BSFN) on the server by passing it parameters.
- Invoke a Business Function (BSFN) on the server using Transaction Processing.
- Invoke a UBE by name.
- Check a UBE call's status asynchronously.
- MBF Poll Events.
- Log out.

Creating a new Mule Project

Create a new Mule Project with Mule Server 4.6.0 EE or greater as runtime:



Project Dependencies

To include the Atina JDE Connector in your Mule application, update your pom.xml file with the following configuration:

Repository Configuration

Add the Atina Repository to the <repositories> section:

```
<repository>
  <id>atina-central</id>
  <name>libs-release</name>
  <url>https://jfrog.atina-connection.com/artifactory/libs-release</url>
</repository>
```

Dependency Declaration

Then, include the connector dependency in the <dependencies> section:

```
<dependency>
```

```
<groupId>com.atina-connection</groupId>
<artifactId>mule-jde-atina-connector</artifactId>
<version>3.0.0</version>
<classifier>mule-plugin</classifier>
</dependency>
```

NOTE

Make sure to replace the version number with the latest available release if a newer one exists.

Other Considerations

To redirect the *JD Edwards EnterpriseOne™* Logger to Mule Logger (allowing you to see the JDE activity in both Console and JDE files defined in the *jdelog.properties*, you may add the following *Async Loggers* to *log4j2.xml* file.

```
<AsyncLogger name="org.mule.modules.jde" level="DEBUG" />
```

Troubleshooting

If you are experiencing issues resolving all project dependencies, follow these steps:

1. Close Anypoint Studio completely.
2. From the project root directory, run the following command in your terminal or command prompt:

```
mvn clean package
```

NOTE

This process ensures that Maven updates the local cache and properly indexes the project upon reopening.

Additional Tips

If the issue persists:

1. Force Maven to update all snapshots and releases:

```
mvn clean package -U
```

1. Delete the local repository cache (□□ this will require re-downloading all dependencies):

```
rm -rf ~/.m2/repository
```

1. After clearing the cache, rerun the build:

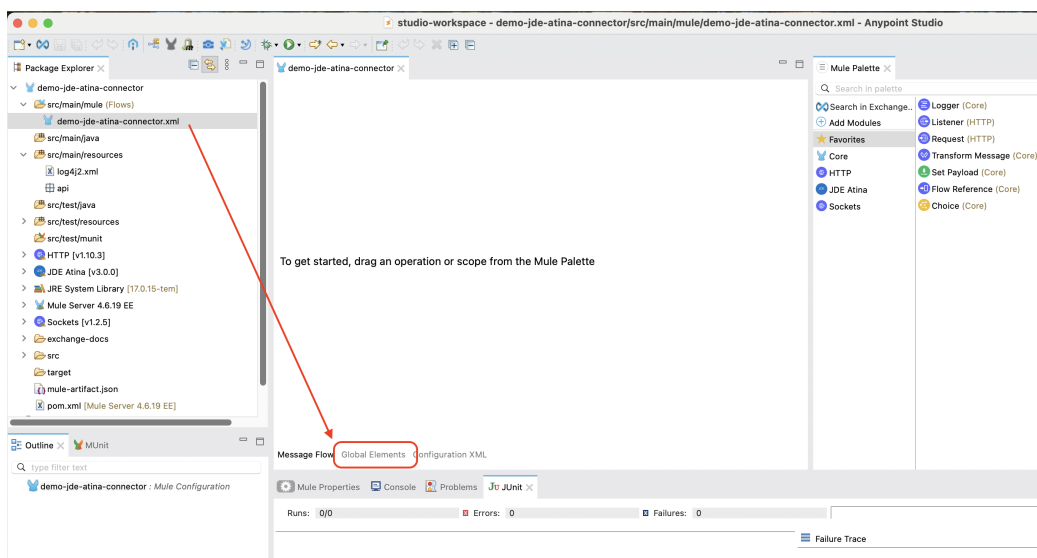
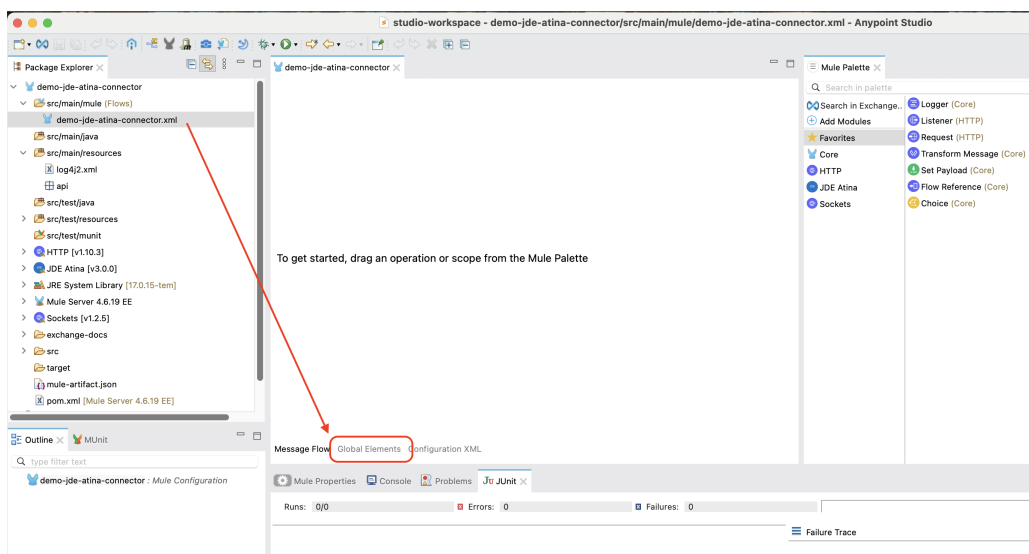
```
mvn clean package
```

3. Reopen Anypoint Studio and verify that all dependencies are now resolved.

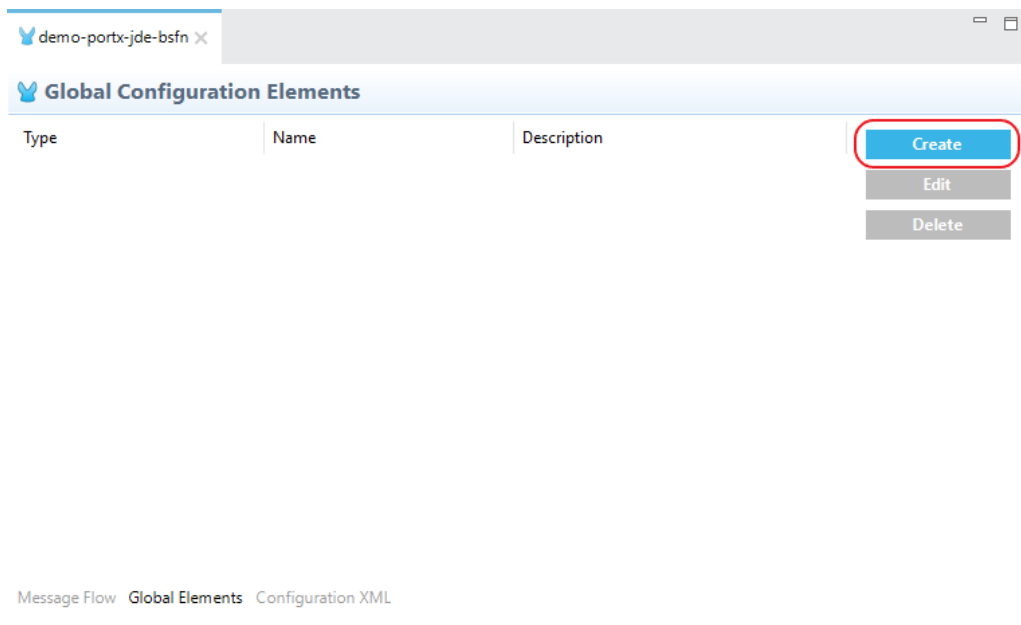
Configure the Global Element

To use the *Atina JDE Connector* in your Mule application, you must configure a global element that can be used by the connector (read more about Global Elements).

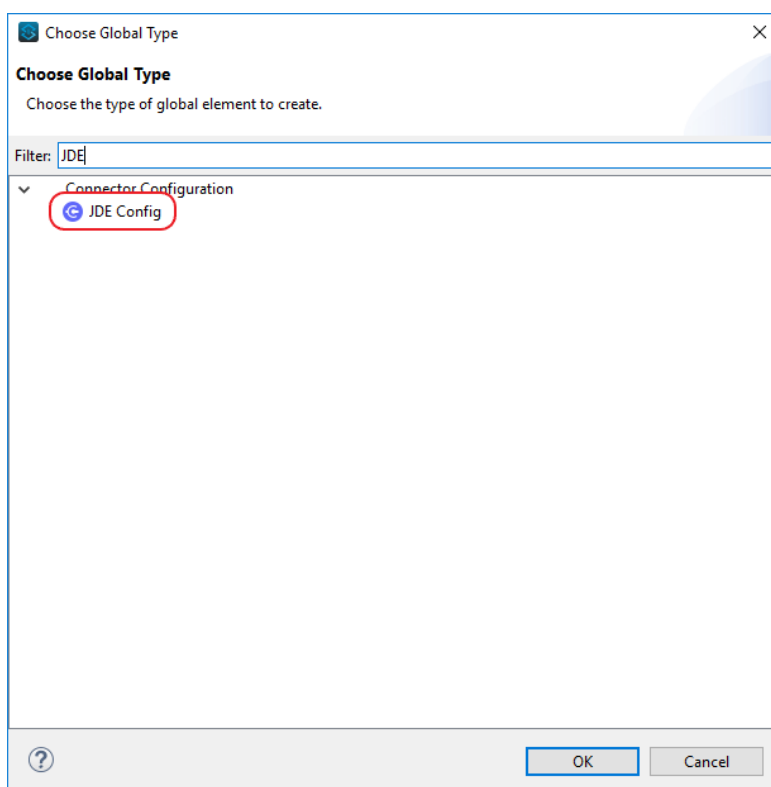
Open the Mule flow for the project, and select the Global Elements tab at the bottom of the Editor Window.



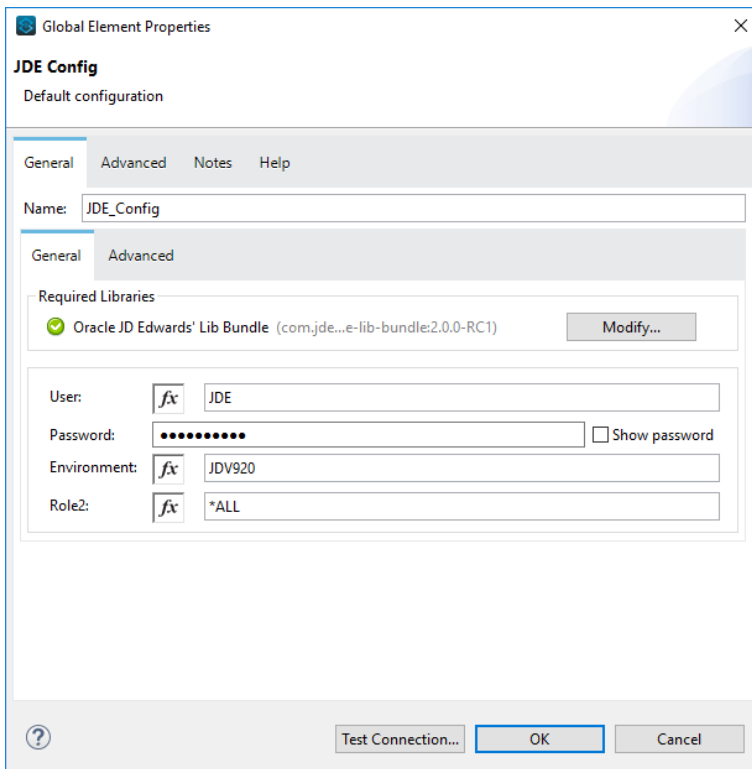
Click Create



Type "JDE" in the filter edit box, and select "JDE Config". Click OK



On the *General* tab, enter the required credential and environment



Global Element Properties

JDE Config
Default configuration

General | Advanced | Notes | Help

Name: JDE_Config

General | Advanced

Required Libraries

- ☒ Oracle JD Edwards' Lib Bundle (com.jde...e-lib-bundle:2.0.0-RC1) Modify...

User: JDE

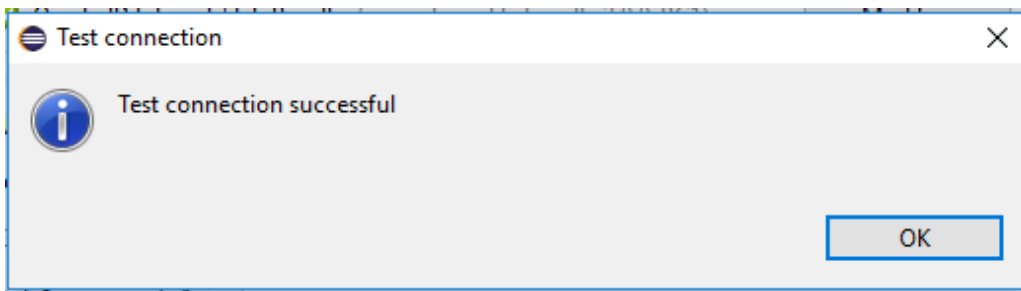
Password: ☐ Show password

Environment: JDV920

Role2: *ALL

? Test Connection... OK Cancel

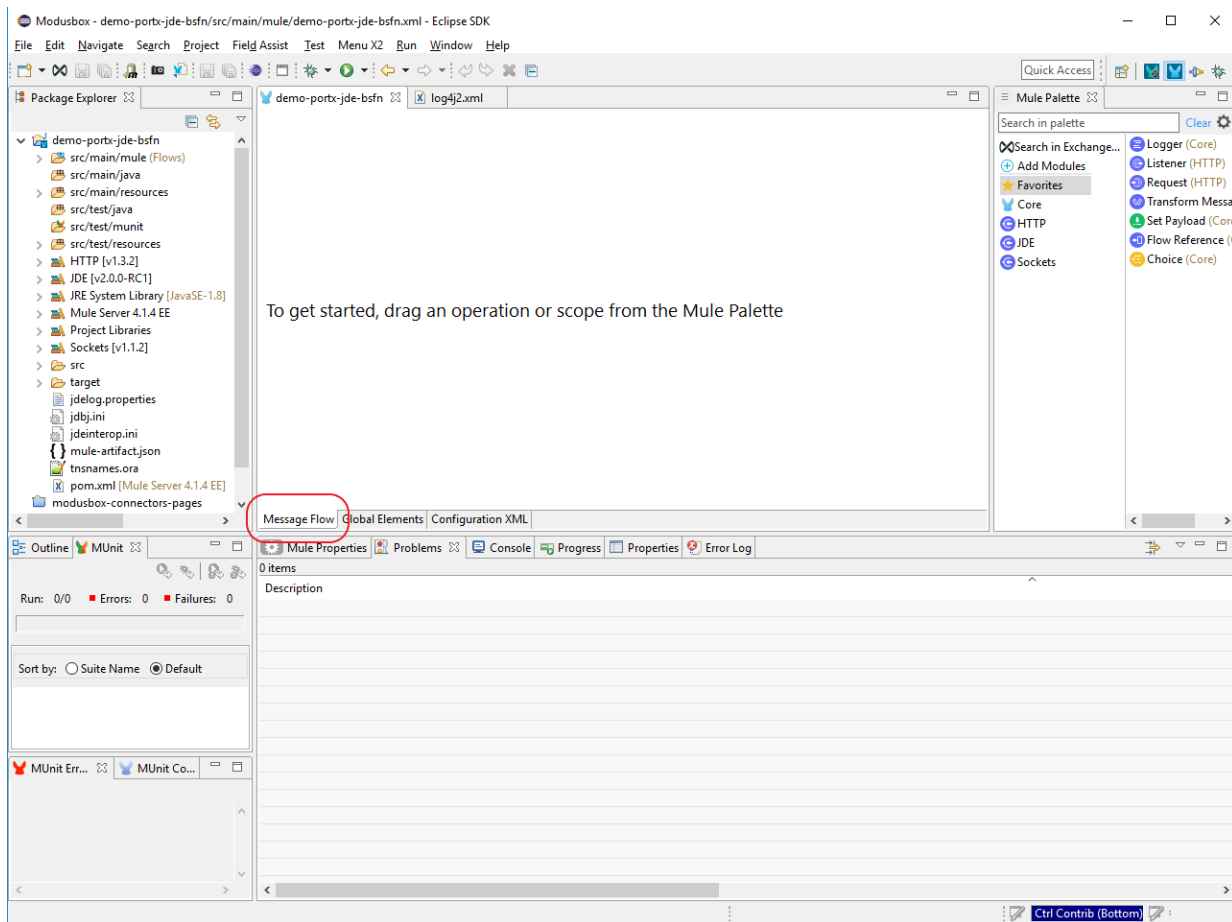
Click *Test Connection*. You should see the following message appear.



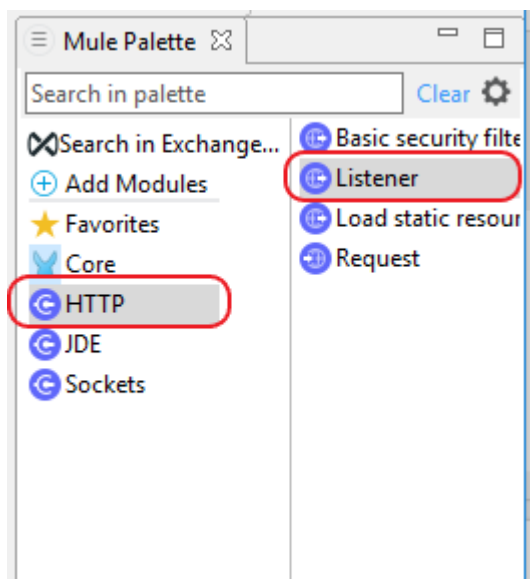
Creating a HTTP Listener for your flow

NOTE : This use case example will create a simple flow to get the address book name from the Address Book table (A/B) invoking the Master Business Function (MBF) on *Oracle's JDE EnterpriseOne* Server.

Go back to the *Message Flow* tab



From the Mule Palette (typically top right), select *HTTP*, and drag Listener to the canvas



Select the *HTTP Listener* component from the canvas, and inspect the properties window

Problems Console Progress Properties Error Log

2 issues found ...

Display Name:

Basic Settings

Connector configuration:  

General

Path:

The connector requires a *Connector Configuration*. Click on **Add** to create a connector configuration.

Give the HTTP endpoint a more descriptive name like *get-AddressBookName-http-endpoint* and press **OK** to go back to the global HTTP endpoint dialog box:

Global Element Properties

HTTP Listener config
Configuration element for a HttpListener.

General Notes Help

Name:

General TLS Advanced

Connection

Protocol:

Host:

Port:

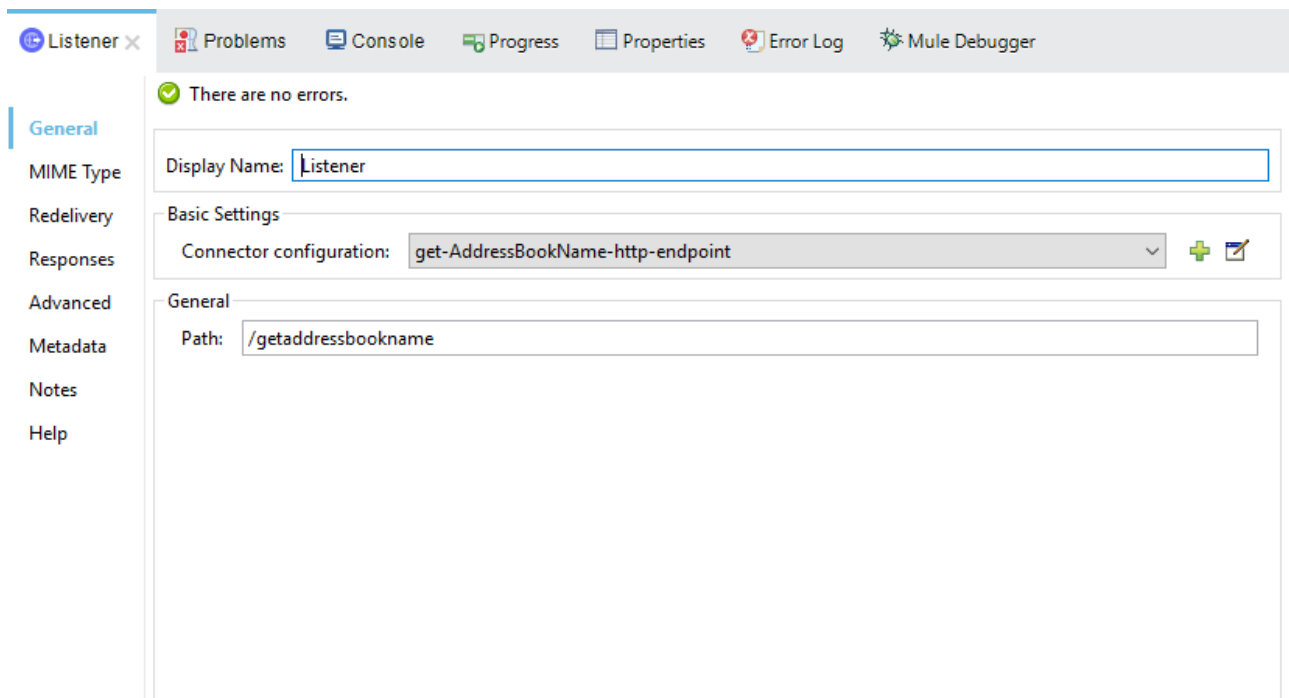
General

Base path:

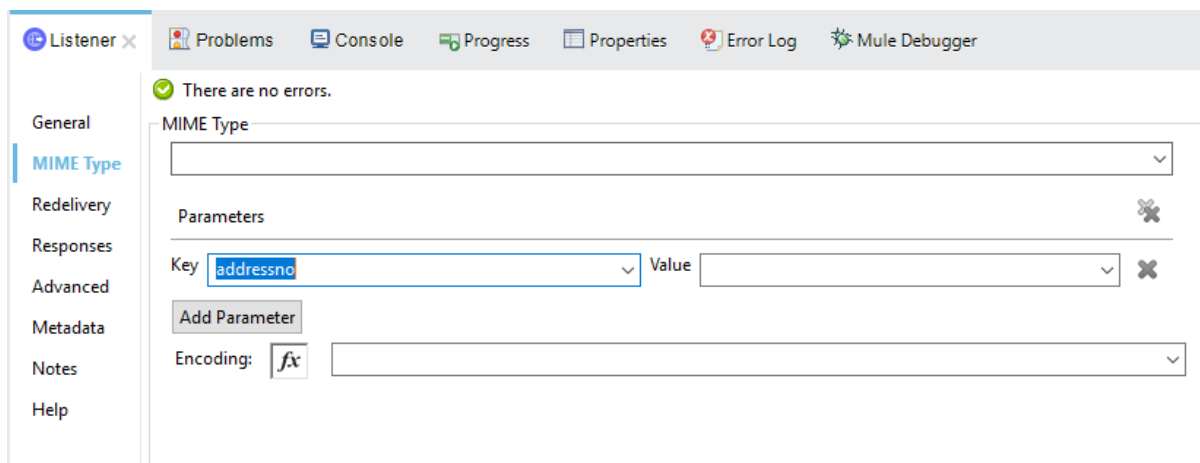
Listener interceptors:



Add a path to the URL eg. *getaddressbookname*.



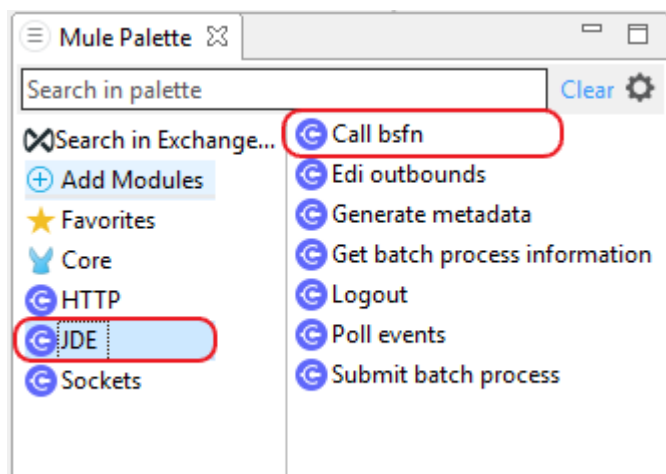
Click on the MIME Type link, and add a parameter for addressno.



Save the project. The connector will be ready to process requests.

Invoke a Business Functions

Locate the **JDE** Connector, and select Call BSFN. Drag this to the canvas.



Drag the connector over to the canvas. Select it and review the properties window. Give it a meaningful name eg. Call AddressBookMasterMBF.

Under the General section, click on the drop-down for Business Function Name.

Attribute 'bsfnName' is required

Display Name: Call AddressBookMasterMBF

Basic Settings

Connector configuration: JDE_Config

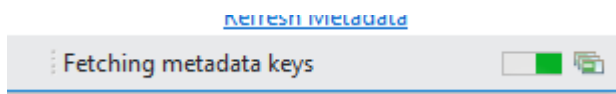
General

Bsfname: [dropdown arrow circled in red]

Input parameters:

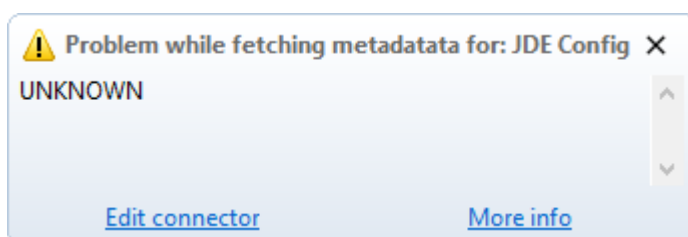
- 1 payload

NOTE : If it is the first time you are selecting a function, this might take a while, as no information has been cached yet. It will need to build a list of all functions available. Please be patient. The status bar (bottom right) will display the following while it is retrieving the metadata.



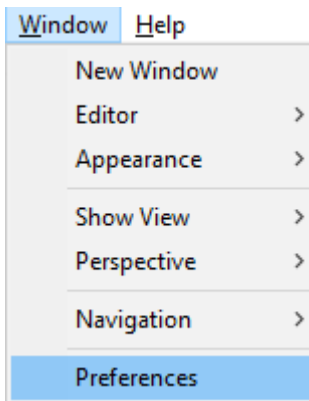
Troubleshooting

If the operation fails (possibly due to a timeout), you will see the below message

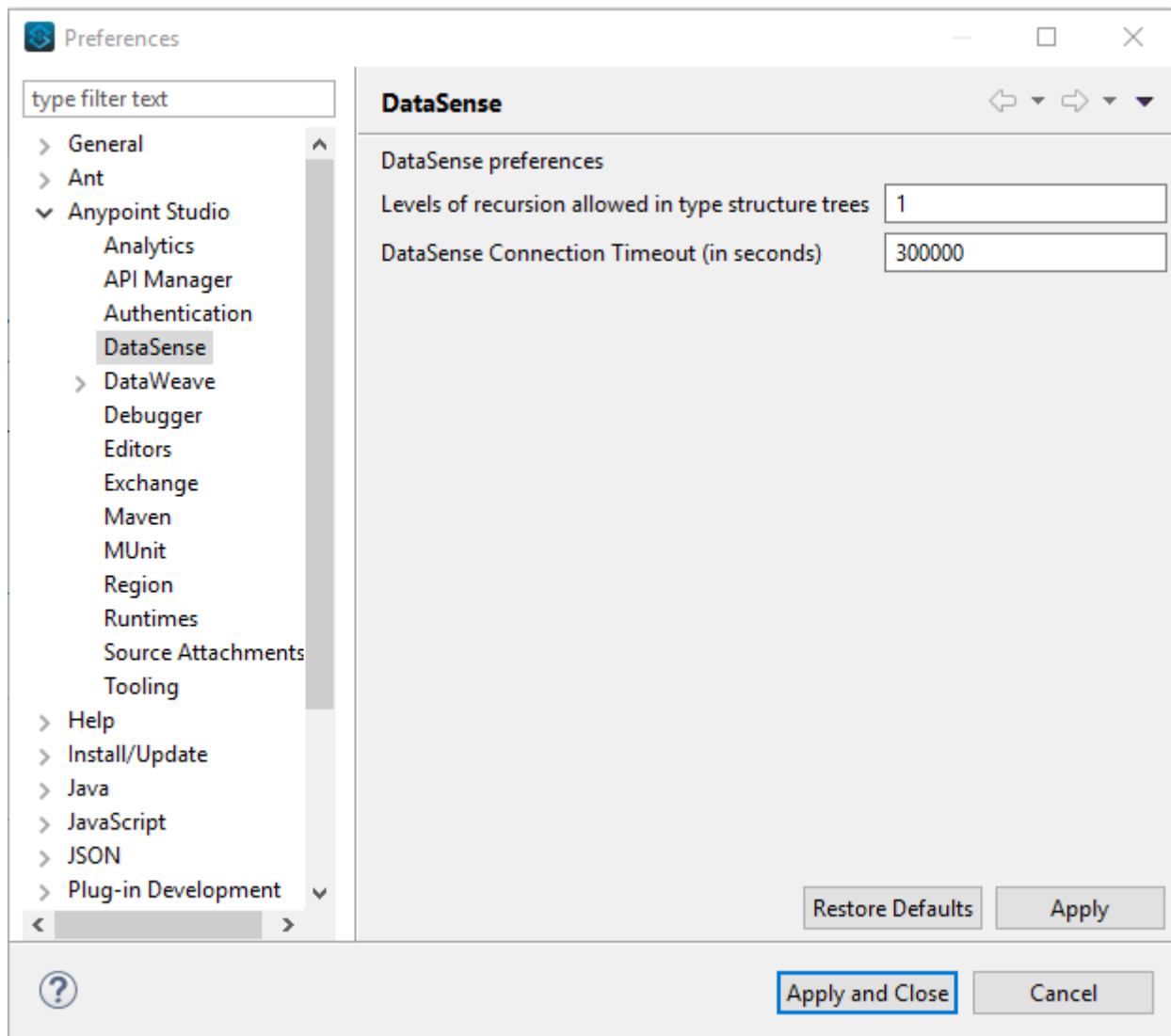


Please review the timeout settings in *Anypoint Studio's* Preferences.

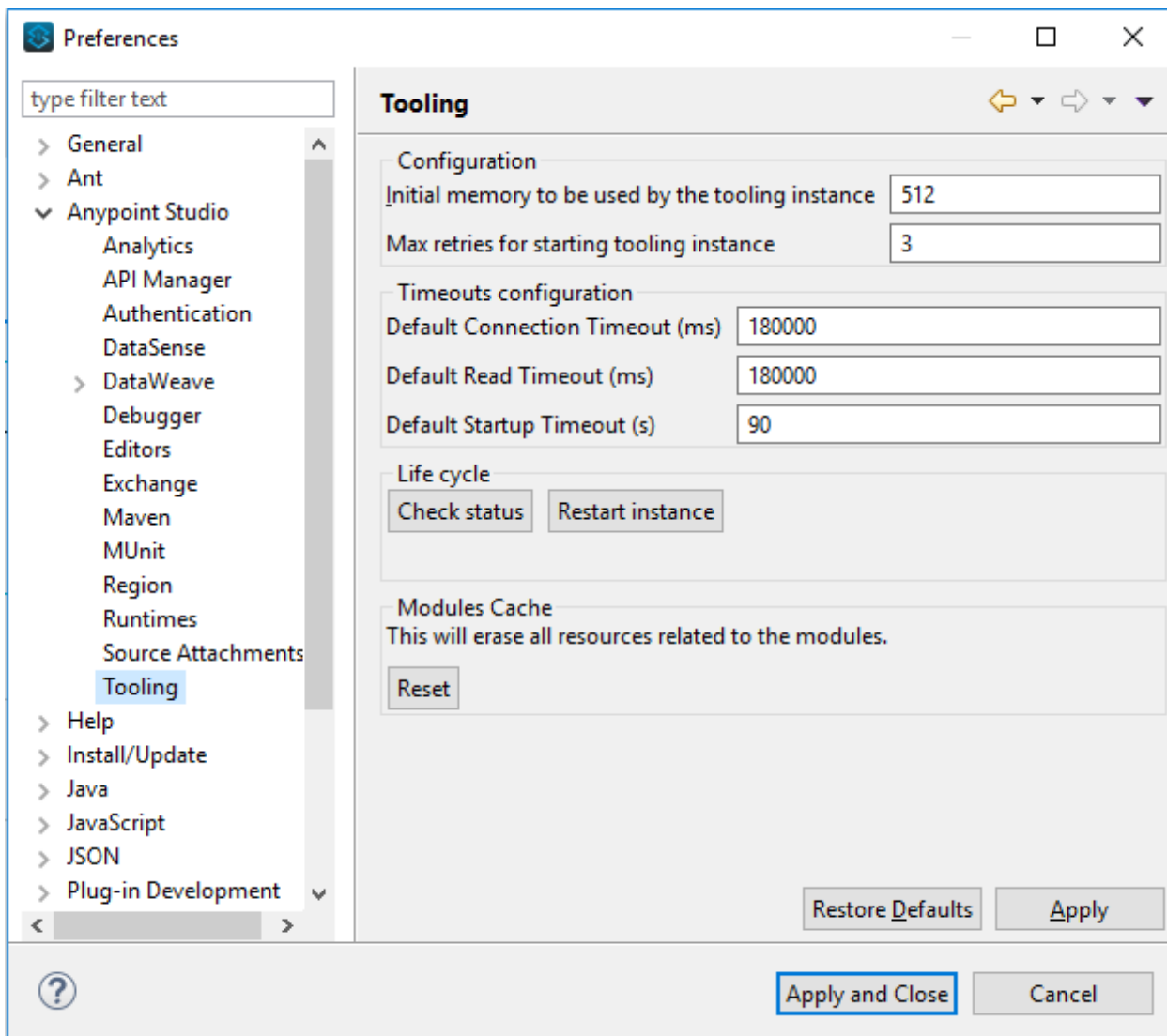
To do this go to the *Window > Preferences* menu



Go to *Anypoint Studio* > *DataSense* and change the *DataSense Connection Timeout* setting as below

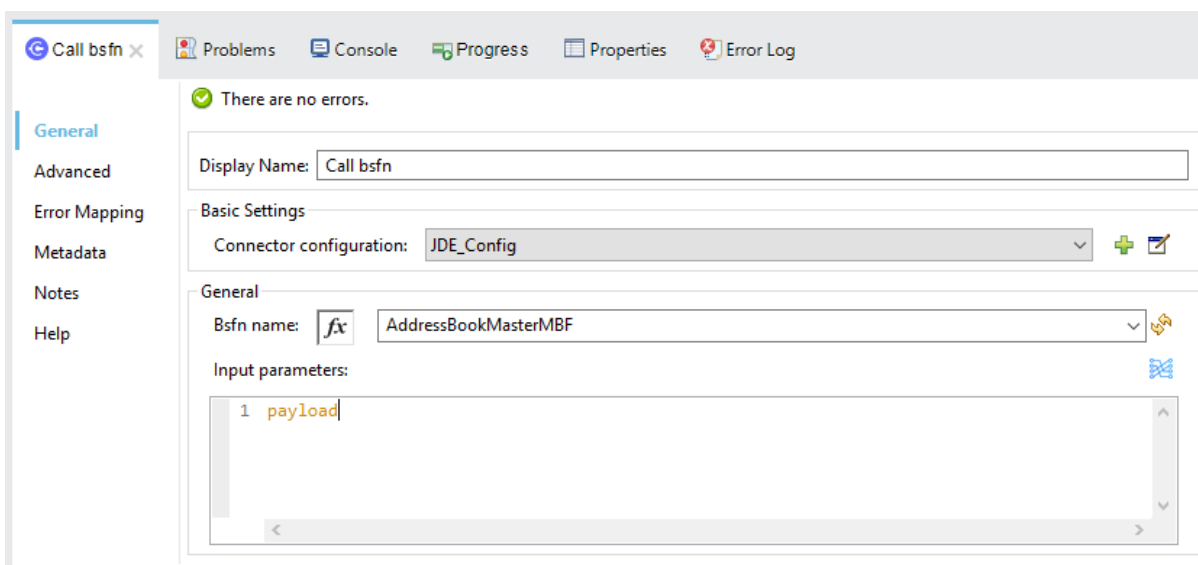


Go to *Anypoint Studio* > *Tooling* and change the *Default Connection Timeout* and *Default Read Timeout* settings as below

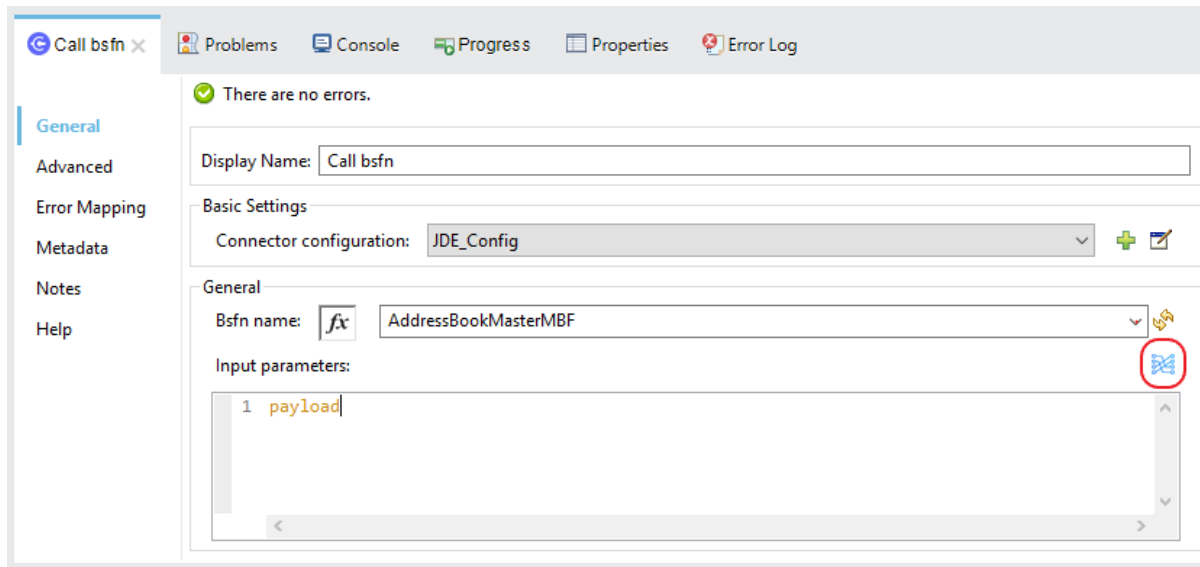


Setting Parameters

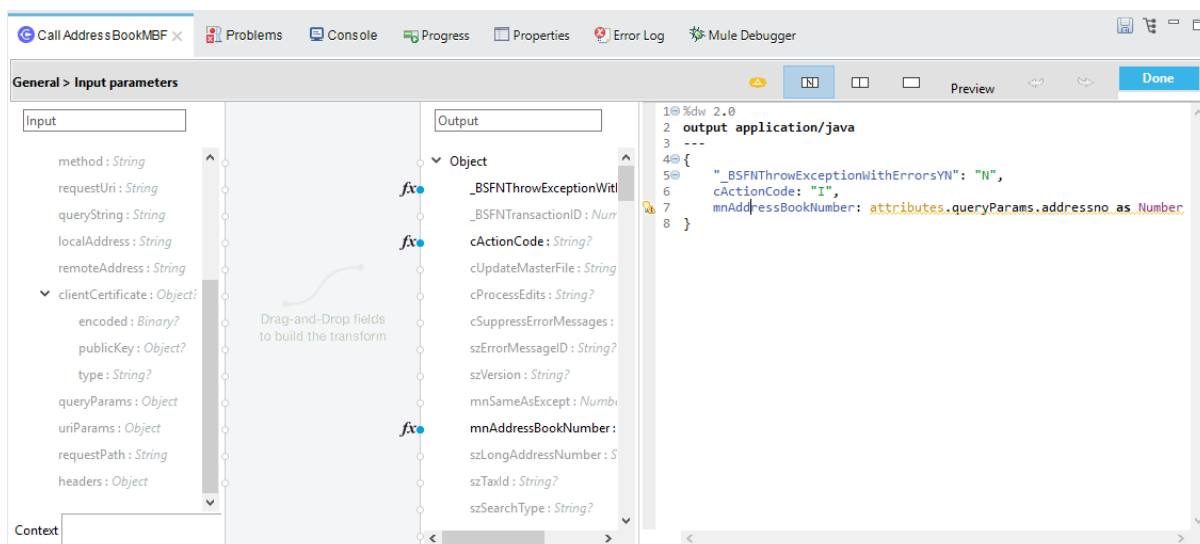
After the system has retrieved the required metadata, select AddressBookMasterMBF from the list. The specification metadata will be retrieved from the enterprise server, and put into the project metadata repository.



You may now assign the input parameters. You can do this by either entering the payload values manually, or via the “Show Graphical View” button.

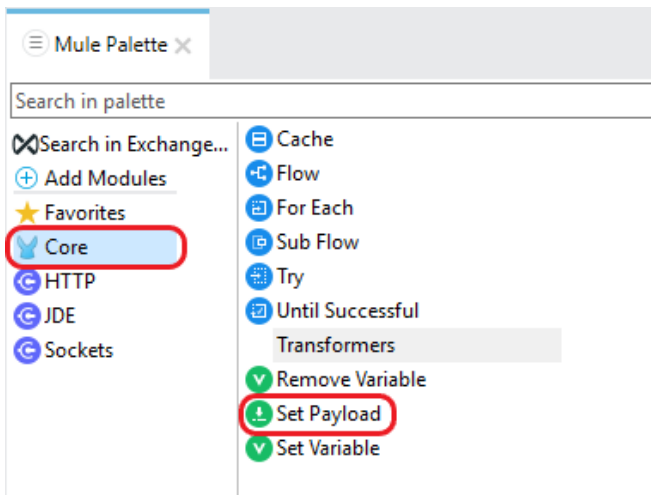


Drag the inputs to outputs, or double-click the output parameter to add to your edit window, and change as required. Be sure to map your query parameter to the function `mnAddressBookNumber`

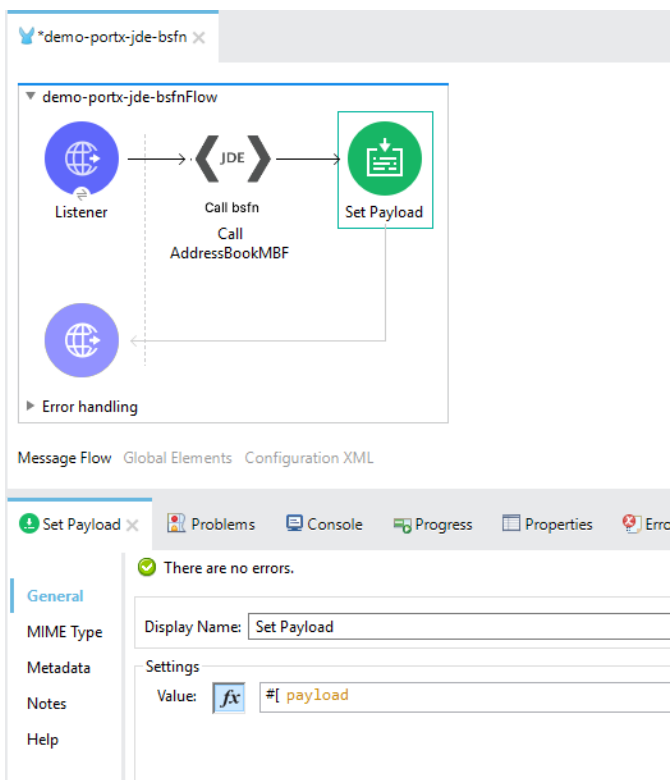


Set Payload Output

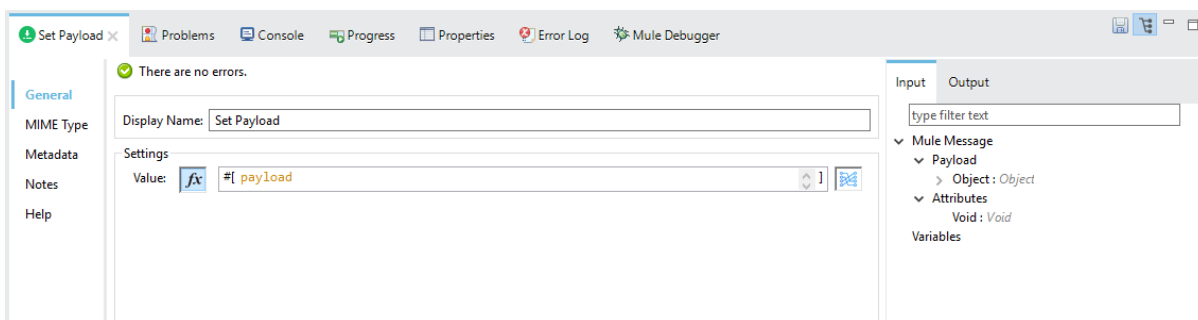
In the Mule Palette, you can either select Core, scroll down to Transformers or type “Payload” in the search bar.



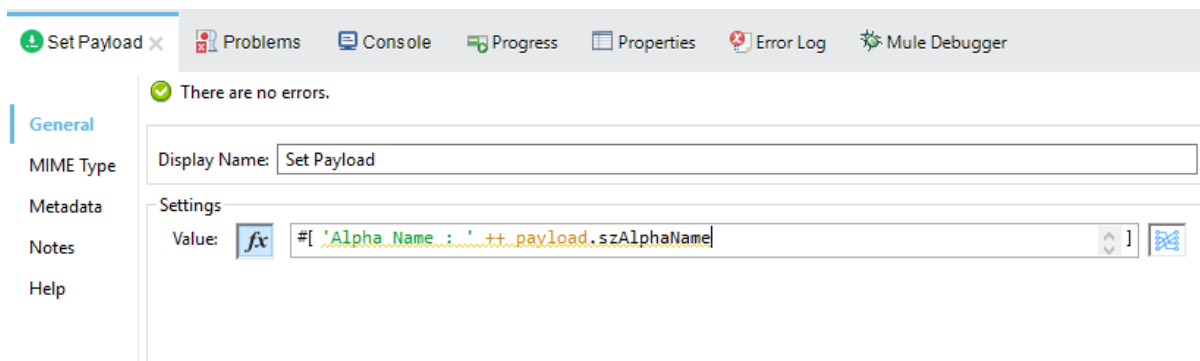
Drag and drop the *Set Payload* to your canvas.



Select the Set Payload component, and review the properties.

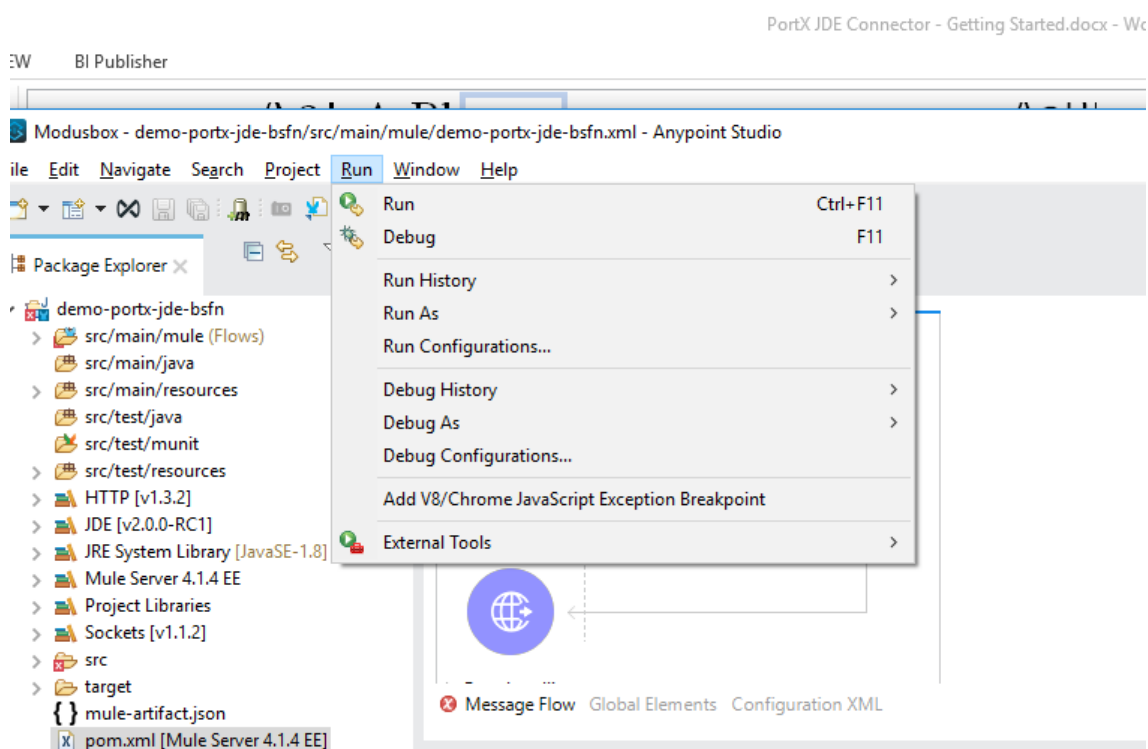


Change the payload to reflect the desired output, and save the project

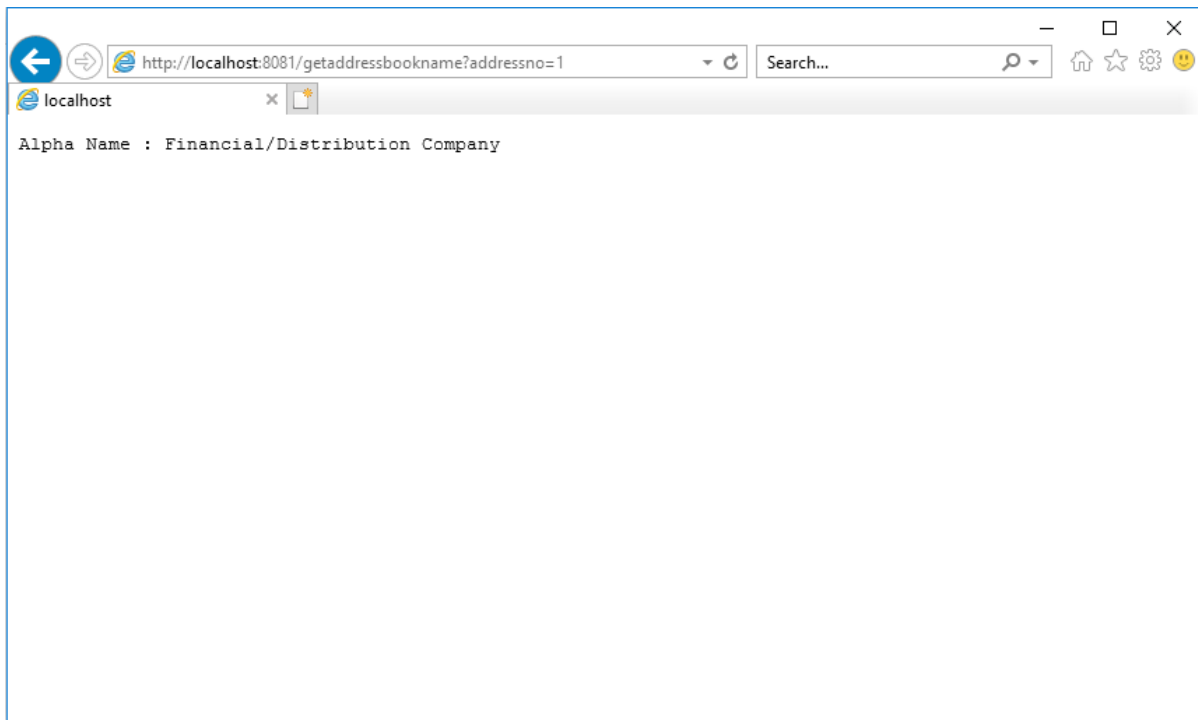


Testing the Mule Flow

To Test your flow, you need to start the Mule application. Go to the *Run* menu, and select *Run*.



After the project has been deployed, you can test you flow by typing the URL into a web browser eg. <http://localhost:8081/getaddressbookname?addressno=1>



PortX JDE Connector – Demo Applications

Example Project

You can download the example source that was created in the Getting Started Guide from

Additional Demos

There are additional demo applications with step by step guides available for download. These cover all the basic operations, and are :

1. [Invoke a Business Function with Transaction Processing](#)
2. [Submit a Batch Process](#)
3. [Retrieve a Batch Process's Status](#)
4. [Poll MBF Events](#)
5. [Poll EDI Events](#)

Appendix A : Required JARs by Tools Release

Tools Release 8.98

Copy these files:

- ApplicationAPIs_JAR.jar
- ApplicationLogic_JAR.jar

- Base_JAR.jar
- BizLogicContainer_JAR.jar
- BizLogicContainerClient_JAR.jar
- BusinessLogicServices_JAR.jar
- castor.jar
- commons-httpclient-3.0.jar
- commons-logging.jar
- Connector.jar
- EventProcessor_JAR.jar
- Generator.jar
- j2ee1_3.jar
- JdbjBase_JAR.jar
- JdbjInterfaces_JAR.jar
- JdeNet_JAR.jar
- jmxremote.jar
- jmxremote_optional.jar
- jmxri.jar
- log4j.jar
- ManagementAgent_JAR.jar
- Metadata.jar
- MetadataInterface.jar
- PMApi_JAR.jar
- Spec_JAR.jar
- System_JAR.jar
- SystemInterfaces_JAR.jar
- xmlparserv2.jar

Tools Releases 9.1 prior Update 4

Copy these files:

- ApplicationAPIs_JAR.jar
- ApplicationLogic_JAR.jar
- Base_JAR.jar
- BizLogicContainer_JAR.jar
- BizLogicContainerClient_JAR.jar

- BusinessLogicServices_JAR.jar
- castor.jar
- commons-httpclient-3.0.jar
- commons-logging.jar
- Connector_JAR.jar
- EventProcessor_JAR.jar
- Generator.jar
- JdbjBase_JAR.jar
- JdbjInterfaces_JAR.jar
- JdeNet_JAR.jar
- jmxremote.jar
- jmxremote_optional.jar
- jmxri.jar
- ManagementAgent_JAR.jar
- Metadata.jar
- MetadataInterface.jar
- PMApi_JAR.jar
- Spec_JAR.jar
- System_JAR.jar
- SystemInterfaces_JAR.jar
- xmlparserv2.jar

Tools Releases 9.1 Update 4 and later updates

Copy these files:

- ApplicationAPIs_JAR.jar
- ApplicationLogic_JAR.jar
- Base_JAR.jar
- BizLogicContainer_JAR.jar
- BizLogicContainerClient_JAR.jar
- BusinessLogicServices_JAR.jar
- castor.jar
- commons-codec.jar
- commons-lang2.6.jar
- commons-logging.jar

- Connector.jar
- EventProcessor_JAR.jar
- Generator_JAR.jar
- httpclient.jar
- httpcore.jar
- httpmime.jar
- j2ee1_3.jar
- JdbjBase_JAR.jar
- JdbjInterfaces_JAR.jar
- JdeNet_JAR.jar
- jmxremote.jar
- jmxremote_optional.jar
- jmxri.jar
- ManagementAgent_JAR.jar
- Metadata.jar
- MetadataInterface.jar
- PMApi_JAR.jar
- Spec_JAR.jar
- System_JAR.jar
- SystemInterfaces_JAR.jar
- xml-apis.jar
- xmlparserv2.jar

Tools Releases 9.2 and later

Copy these files:

- ApplicationAPIs_JAR.jar
- ApplicationLogic_JAR.jar
- Base_JAR.jar
- BizLogicContainer_JAR.jar
- BizLogicContainerClient_JAR.jar
- BusinessLogicServices_JAR.jar
- commons-codec.jar
- castor.jar
- commons-lang2.6.jar

- commons-logging.jar
- Connector.jar
- EventProcessor_JAR.jar
- Generator.jar
- httpclient.jar
- httpcore.jar
- httpmime.jar
- j2ee1_3.jar
- JdbjBase_JAR.jar
- JdbjInterfaces_JAR.jar
- JdeNet_JAR.jar
- jmxremote.jar
- jmxremote_optional.jar
- jmxri.jar
- ManagementAgent_JAR.jar
- Metadata.jar
- MetadataInterface.jar
- PMApi_JAR.jar
- Spec_JAR.jar
- System_JAR.jar
- SystemInterfaces_JAR.jar
- xerces.jar
- xml-apis.jar
- xmlparserv2.jar

Appendix B : Additional Information

Common Call Business Function parameters

There is four common parameters used to invoke a Business Function:

Parameter	Input/Output	Type	Description
-----------	--------------	------	-------------

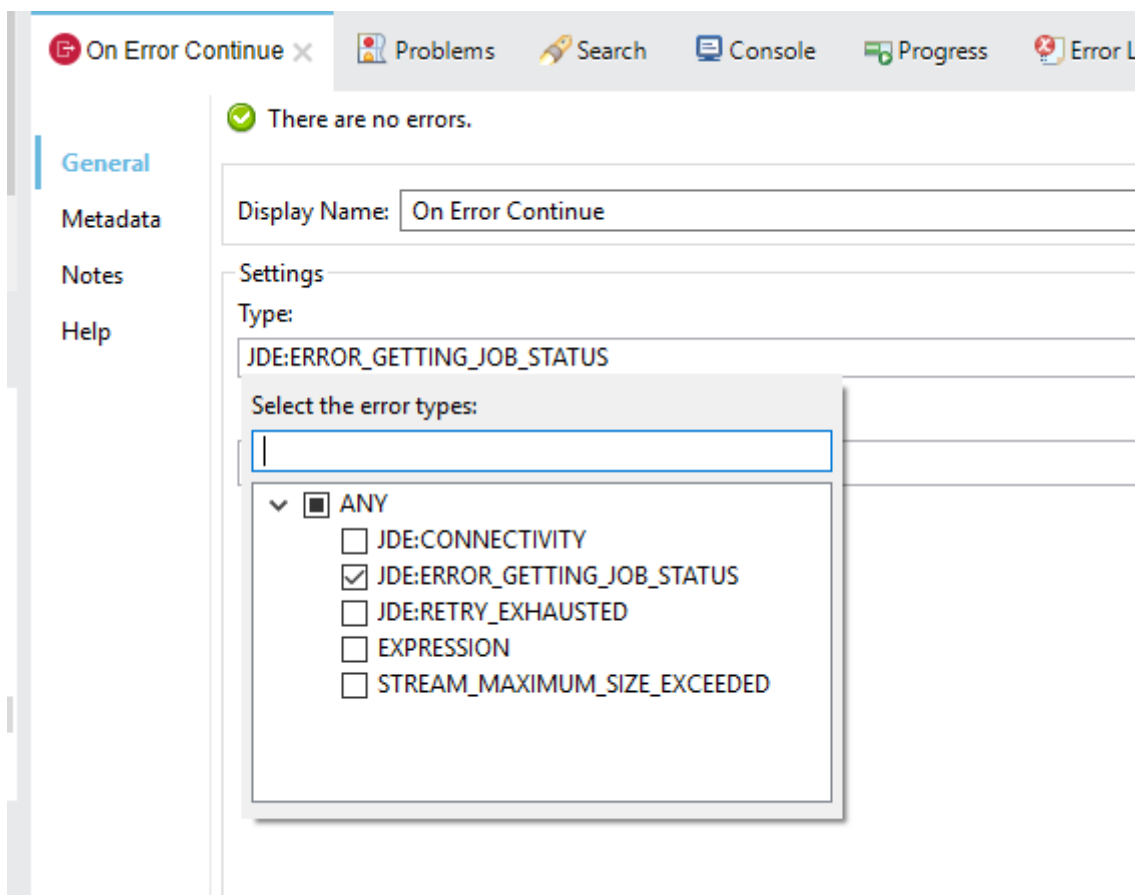
_BSFNThrowException WithErrorsYN	Input	STRING	Values: Y, The flow will throw Runtime Exception with cause: org.mule.modules.jde.exceptions.CallBSFNException.
_BSFNReturnCode	Output	INTEGER	Values: 0: Processed Correctly, 1: There is warnings, 2: There is errors.
_BSFNNumberOfErrors	Output	INTEGER	Number of errors
_BSFNDetailErrors	Output	STRING	Detail of the error. It return a XML representation of CallObjectErrorList

Usage :

Handling Errors

In Mule 4, all thrown errors will be displayed in the *Type* selection of the Error Handler. Simple select the errors you want to handle, and process them.

NOTE : All JD Edwards related errors will start with *JDE*:



This is an example of CallObjectErrorItem object:

```
<com.jdedwards.system.kernel.JdeNetCallObjectErrorList>
  <mErrors>
    <com.jdedwards.system.kernel.CallObjectErrorItem>
      <mErrorId>0</mErrorId>
      <mDDItem>1212</mDDItem>
      <mLineNumber>315</mLineNumber>
      <mFileName>b0100094.c</mFileName>
      <mSubText>&#x0;</mSubText>
      <mAlphaDescription>Error: Address Number Already Assigned</mAlphaDescription>
      <mGlossaryText>CAUSE . . . . The Address Number entered is already
assigned.&#xd;
      RESOLUTION. . Enter an Address Number that is not already assigned.&#xd;
      </mGlossaryText>
      <mErrorLevel>1</mErrorLevel>
    </com.jdedwards.system.kernel.CallObjectErrorItem>
    <com.jdedwards.system.kernel.CallObjectErrorItem>
      <mErrorId>11</mErrorId>
      <mDDItem>018A</mDDItem>
      <mLineNumber>544</mLineNumber>
      <mFileName>rtk_ddvl.c</mFileName>
      <mSubText>Search Type|Y|01|ST&#x0;</mSubText>
      <mAlphaDescription>Error: Y not found in User Defined Code 01
ST&#x0;</mAlphaDescription>
      <mGlossaryText>CAUSE . . . . Search Type Y was not found in User Defined
Code&#xd;
      for system 01 , type ST&#x0; .&#xd;
      RESOLUTION. . Enter a valid Search Type or use Visual Assist to
search&#xd;
      for a valid value.
      </mGlossaryText>
      <mErrorLevel>1</mErrorLevel>
    </com.jdedwards.system.kernel.CallObjectErrorItem>
  </mErrors>
  <mBsfncErrorcode>2</mBsfncErrorcode>
</com.jdedwards.system.kernel.JdeNetCallObjectErrorList>
```

Defining Data Selection

- The parameter *Selection* is used to define UBE Data Selection.
- The sentence is similar to a WHERE clause of an SQL statement.
- The *Selection* syntax is:
 - table.column_name operator [value | table.column_name];
- The table must be a JDE table that belongs to the main view of the UBE.
- Column Name must be a JDE Data Item Alias.

- The following operators can be used in the *Selection* :

Operator	Description
=	Equal
<>	Not equal
<>	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
≤	Less than or equal
BETWEEN	Between an inclusive range
NOT BETWEEN	Not Between an exclusive range
IN	To specify multiple possible values for a column
NOT IN	To exclude multiple possible values for a column

- The values can be literals or other table columns.
- Literals can be String or Number
- The sentence can include the AND and/or the OR conditions
- To override the default precedence you need to use parenthesis as
 - C1 AND (C2 OR C3)
 - The sentence only accept one level of Parenthesis.

For example, this is a valid sentence because the maximum level of Parenthesis opened is 1.

C1 AND (C2 OR C3) AND (C4 OR C5)

otherwise, this is an invalid sentences because the maximum level of Parenthesis opened is 2.

C1 AND (C2 OR (C3 AND C4))

Examples:

```

F4211.KC00 = '00001' AND F4211.DOC0 > 10332
F4211.KC00 = '00001' AND F4211.DOC0 >= 10332
F4211.KC00 = '00001' AND F4211.DOC0 <= 10332
F4211.KC00 = '00001' AND F4211.DOC0 <> 10332
F4211.KC00 = '00001' AND ( F4211.DCTO = 'SO' OR F4211.DCTO = 'SI' )
F4211.KC00 = '00001' AND F4211.DCTO IN ('SO','SI')
F4211.KC00 = '00001' AND F4211.DCTO NOT IN ('SO','SI')
F4211.KC00 = '00001' AND F4211.DOC0 BETWEEN 1022 AND 400
F4211.KC00 = '00001' AND F4211.DOC0 NOT BETWEEN 1022 AND 400
F4211.MCU = F4211.EMCU AND F4211.DOC0 NOT BETWEEN 1022 AND 400

```